

Universidad del Valle de Guatemala

Facultad de Ingeniería · Departamento de Ciencias de la Computación

CC3067 Redes · Sección 10

Laboratorio 5

Mini datacenter virtualizado con QEMU/KVM

WordPress en arquitectura de 2 capas con HTTPS, SFTP y administración remota

Integrante	Carné
Nicolás Concuá	23197
Daniela Ramírez	23053
Fernando Mendoza	19644

Guatemala, 10 de noviembre de 2026

Repositorio: github.com/nicoCT04/Lab5-Redes

Resumen

Se implementó un sitio WordPress separado en dos capas sobre dos máquinas virtuales con Ubuntu Server 26.04.1 LTS, ejecutadas con **QEMU 8.2.2 y aceleración KVM** dentro de WSL2 en un equipo con Windows 11. La VM `lab5-web` aloja Nginx, PHP-FPM y WordPress; la VM `lab5-db` aloja MySQL 8.4 LTS. Ambas se comunican por una red privada 192.168.56.0/24 implementada con un puente Linux y se administran exclusivamente por SSH desde el host, incluida la gestión de la base de datos. El sitio se publica solo por HTTPS con un certificado de Let's Encrypt válido para el navegador y se accede por el nombre `redes-lab5-wp.duckdns.org`, que el host resuelve localmente con su archivo hosts.

La seguridad se diseñó por capas: servicios en puertos no conocidos (SSH 22022, MySQL 33306), firewall UFW con denegación por defecto en ambas VM, una base de datos que solo acepta conexiones del servidor web, un usuario de MySQL limitado a la base de WordPress y un único usuario (`wpadmin`) con escritura sobre el sitio, al que se accede por SFTP enjaulado. El certificado se obtuvo con el reto DNS-01, el único método de Let's Encrypt compatible con un servidor que no es accesible desde Internet.

Toda la infraestructura se crea y se configura con scripts versionados: un comando prepara el host, otro crea las VM y otro despliega el laboratorio completo y registra las evidencias. Las pruebas de la sección 8 cubren cada requisito del enunciado, incluidos los casos negativos.

Tabla 1. *Cumplimiento de requisitos.*

Requisito del enunciado	Resultado	Evidencia
2 VM con Ubuntu Server 26.04 LTS sin GUI	Cumplido · 26.04.1 LTS, sin pantalla	Fig. 1, 2
Capa web con cliente MySQL	Cumplido · Nginx + PHP-FPM, mysql 8.4.11	Fig. 12, 13
BD en VM aparte, usuario propio de WordPress	Cumplido · wp_user sin privilegios globales	Fig. 10, 11
HTTPS con certificado de Let's Encrypt	Cumplido · válido hasta 08/02/2027 (UTC)	Fig. 21-23
SFTP solo para el administrador designado	Cumplido · wpadmin, chroot /var/www	Fig. 15-19
SSH y MySQL en puertos no conocidos	Cumplido · 22022 y 33306	Fig. 8, 11
UFW habilitado solo con puertos usados	Cumplido · ambas VM	Fig. 9
Administración remota (incluida la BD)	Cumplido · solo SSH, sin consola	Fig. 8, 11
FileZilla en el host	Cumplido · FileZilla 3.71.1	Fig. 16-18
Acceso por nombre de dominio local	Cumplido · archivo hosts	Fig. 23

Índice

Resumen 2
1. Introducción y objetivos 4
2. Hipervisor: QEMU/KVM 5
2.1 Justificación 5
3. Versiones de software 6
4. Arquitectura y direccionamiento 8
4.1 Decisiones de diseño 9
5. Procedimiento por etapas 11
5.1 Preparación del host 11
5.2 Creación de las máquinas virtuales 12
5.3 Windows como equipo cliente 12
5.4 Acceso SSH y puerto no conocido 12
5.5 Firewall UFW 13
5.6 Capa de base de datos 14
5.7 Capa web 15
5.8 Permisos con chown y chmod 16
5.9 SFTP y FileZilla 17
5.10 HTTPS con Let's Encrypt 19
5.11 Resolución de nombres local 21
6. Cliente y servidor 23
7. Seguridad: defensa en profundidad 23
8. Pruebas de funcionamiento 25
9. Problemas encontrados y soluciones 28
10. Conclusiones 28
11. Referencias 29
Anexo A. Reproducción del laboratorio 30

1. Introducción y objetivos

El laboratorio plantea montar un servicio web con los componentes de un centro de datos pequeño: servidores virtualizados, una red interna, separación entre aplicación y datos, acceso administrativo remoto y publicación cifrada. El enunciado no entrega comandos; cada herramienta y versión se eligió y se justifica en este documento.

Tabla 2. *Objetivos de la actividad y su desarrollo.*

Objetivo	Cómo se aborda	Sección
Permisos de archivos (chown/chmod)	Modelo propietario/grupo/otros que deja la escritura solo a wpadmin y la lectura a www-data	5.8
Gestión remota por SSH	Ambas VM y MySQL se administran solo por SSH con llave; las VM no tienen pantalla	5.4, 5.6
Cliente y servidor	Cada conexión se analiza como aplicación y como ordenador; la VM web es cliente de la BD	6
Resolución de nombres local	El archivo hosts del host tiene prioridad sobre el DNS y dirige el dominio a la VM	5.11
Servicio de 2 capas seguro	Capas separadas, firewall, puertos no conocidos, TLS y mínimo privilegio	4, 7

2. Hipervisor: QEMU/KVM

Se usó **KVM** (*Kernel-based Virtual Machine*) con **QEMU** como monitor de máquinas virtuales. KVM es un módulo del kernel de Linux que usa las extensiones de virtualización del procesador (Intel VT-x) para ejecutar el código de las VM directamente en el CPU; QEMU crea cada VM como un proceso, emula los dispositivos y expone discos y tarjetas de red *virtio*, diseñadas para virtualización. Es la misma base que usan Proxmox VE y OpenStack.

KVM suele clasificarse como hipervisor de tipo 1, porque convierte al propio kernel en hipervisor. En este equipo el kernel con KVM es el de WSL2, que a su vez corre sobre Hyper-V en Windows 11 con virtualización anidada. En la práctica las VM del laboratorio se ejecutan en un segundo nivel de virtualización, con aceleración por hardware en ambos niveles.

2.1 Justificación

Tabla 3. *Comparación de QEMU/KVM con VirtualBox para este equipo.*

Criterio	QEMU/KVM en WSL2	VirtualBox en el mismo equipo
Convivencia con WSL2/Hyper-V	Nativa: WSL2 expone /dev/kvm con virtualización anidada	Corre sobre Hyper-V (NEM); en pruebas las VM se congelaban
Rendimiento	virtio para disco y red; CPU con -cpu host	Emulación de NIC Intel PRO/1000 y SATA por defecto
Automatización	Línea de comandos completa; VM como procesos sin pantalla	VBoxManage; la instalación requiere ISO y autoinstall
Instalación del SO	Imagen cloud oficial de Ubuntu 26.04 + cloud-init (listo en ~1 min)	Instalador completo (10–20 min por VM)
Relación con la industria	Base de Proxmox, OpenStack y nubes públicas	Orientado a escritorio
Costo y licencia	Software libre (GPL)	Gratuito (GPL; extensiones con licencia propia)

El diseño de red reproduce el de un hipervisor de escritorio: cada VM tiene un adaptador NAT para salir a Internet y un adaptador en una red privada con IP fija, sin DHCP. La tabla 4 muestra la equivalencia.

Tabla 4. *Equivalencia de conceptos de red y operación.*

Concepto	VirtualBox	Implementación con QEMU/KVM
Red privada host ↔ VM	Adaptador host-only 192.168.56.1/24	Puente Linux br-lab5 192.168.56.1/24 (servicio lab5-red)
Conexión de cada VM a la red privada	Adaptador 2 host-only	Interfaz tap (tap-web, tap-db) unida al puente
Salida a Internet	Adaptador 1 NAT, 10.0.2.15	NAT de usuario de QEMU, 10.0.2.15, puerta de enlace 10.0.2.2

Concepto	VirtualBox	Implementación con QEMU/KVM
Nombre de interfaces en la VM	enp0s3 (NAT), enp0s8 (host-only)	Igual: NIC en ranuras PCI 3 y 8
VM sin pantalla	Inicio headless	-display none; salida serie a un archivo
Instalación desatendida	ISO + autoinstall	Imagen cloud + cloud-init (NoCloud)

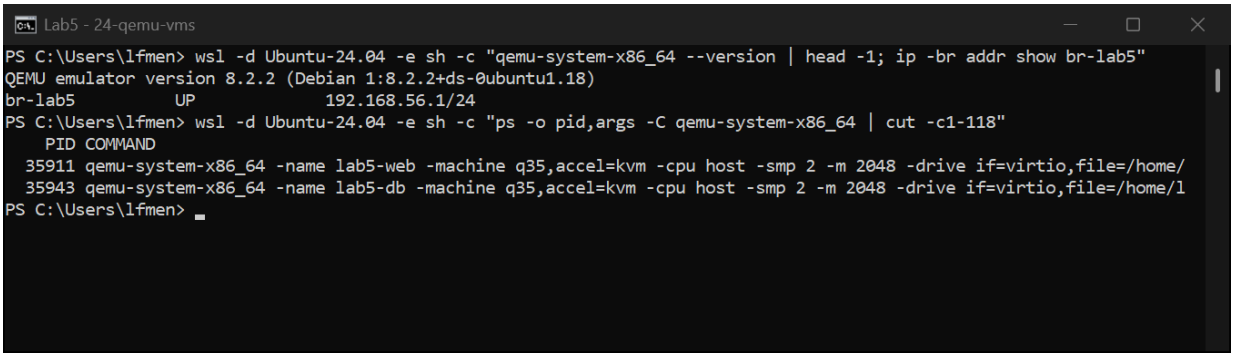


Figura 1. *QEMU 8.2.2, puente br-lab5 y las dos VM ejecutándose con KVM (accel=kvm, -cpu host).*

3. Versiones de software

Las versiones de los servicios provienen de los repositorios oficiales de Ubuntu 26.04: reciben actualizaciones de seguridad durante todo el soporte del sistema sin agregar repositorios de terceros.

Tabla 5. *Versiones utilizadas y criterio de selección.*

Componente	Versión	Justificación
Host	Windows 11 (build 26300) · WSL 2.7.12, kernel 6.18	Equipo de trabajo; WSL2 provee el kernel Linux con KVM
Distribución del host Linux	Ubuntu 24.04.4 LTS (WSL2)	LTS; paquetes de QEMU e iptables del repositorio oficial
Hipervisor	QEMU 8.2.2 + KVM	Ver sección 2
Sistema operativo de las VM	Ubuntu Server 26.04.1 LTS (kernel 7.0)	Requisito; LTS con 5 años de soporte; imagen cloud mínima, sin GUI
Aprovisionamiento	cloud-init 26.1	Hostname, usuario, llave y red declarados en archivos versionables
SSH / SFTP	OpenSSH 10.2p1	Incluido en Ubuntu; chroot y ForceCommand por usuario
Firewall	UFW 0.36.2	Requisito; reglas por puerto y origen sobre nftables
Servidor web	Nginx 1.28.3	Menor uso de memoria que Apache; configuración TLS concisa
Intérprete PHP	PHP-FPM 8.5.4	PHP en un proceso separado, con su propio usuario

Componente	Versión	Justificación
CMS	WordPress 7.1.4 · WP-CLI 2.12.0	Versión estable vigente; instalación sin asistente web
Base de datos	MySQL 8.4.11 LTS	Requisito (MySQL); rama LTS del repositorio de Ubuntu
Certificados	certbot 4.0.0	Cliente ACME de la EFF; hooks para DNS-01
Clientes del host	FileZilla 3.71.1 · Microsoft Edge 156 · OpenSSH de Windows	FileZilla es requisito; versión portable oficial verificada con SHA-512

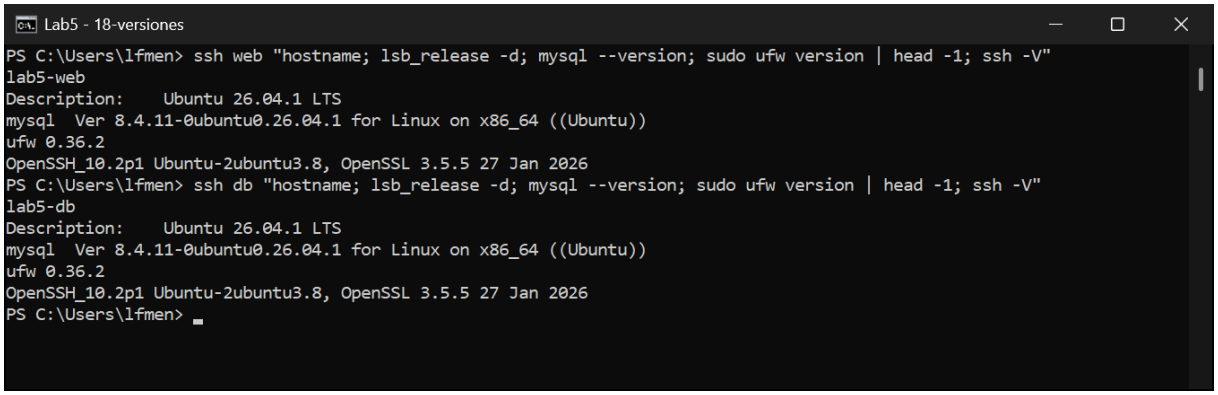


Figura 2. *Versiones del sistema y de los servicios en ambas VM, consultadas por SSH.*

4. Arquitectura y direccionamiento

La capa web y la capa de datos están en VM distintas. Los servicios solo se ofrecen en la red privada 192.168.56.0/24; la salida a Internet de cada VM pasa por la NAT de QEMU y es solo saliente. Windows 11 actúa como equipo cliente: una ruta estática dirige 192.168.56.0/24 hacia WSL, que reenvía el tráfico al puente y lo enmascara con la dirección 192.168.56.1. De esa forma las VM ven al host siempre con la misma IP y las reglas de firewall no dependen de la dirección de WSL, que cambia en cada arranque.

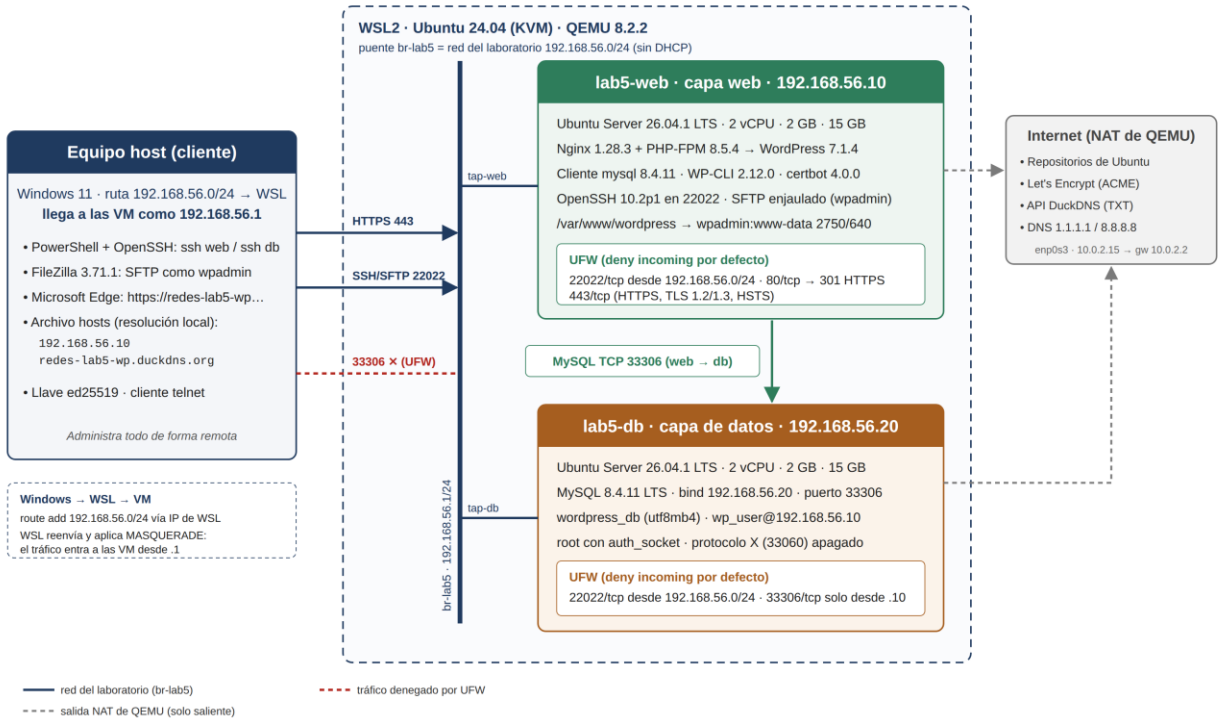


Figura 3. *Diagrama de la implementación: capas de virtualización, direcciones, servicios, puertos y reglas de firewall.*

Tabla 6. *Direccionamiento y puertos.*

Equipo	Nombre / IP	Servicios expuestos (puerto por defecto → usado)
Host Windows 11	Ruta 192.168.56.0/24 → WSL	Ninguno; clientes SSH, SFTP, telnet y navegador
WSL2 (puente br-lab5)	192.168.56.1/24	Ninguno; reenvío y NAT hacia la red del laboratorio
VM capa web	lab5-web · 192.168.56.10/24	SSH/SFTP 22 → 22022; HTTPS 443; HTTP 80 (solo redirige)
VM capa de datos	lab5-db · 192.168.56.20/24	SSH 22 → 22022; MySQL 3306 → 33306; protocolo X 33060 desactivado
Dominio	redes-lab5-wp.duckdns.org	Resuelto a 192.168.56.10 por el archivo hosts

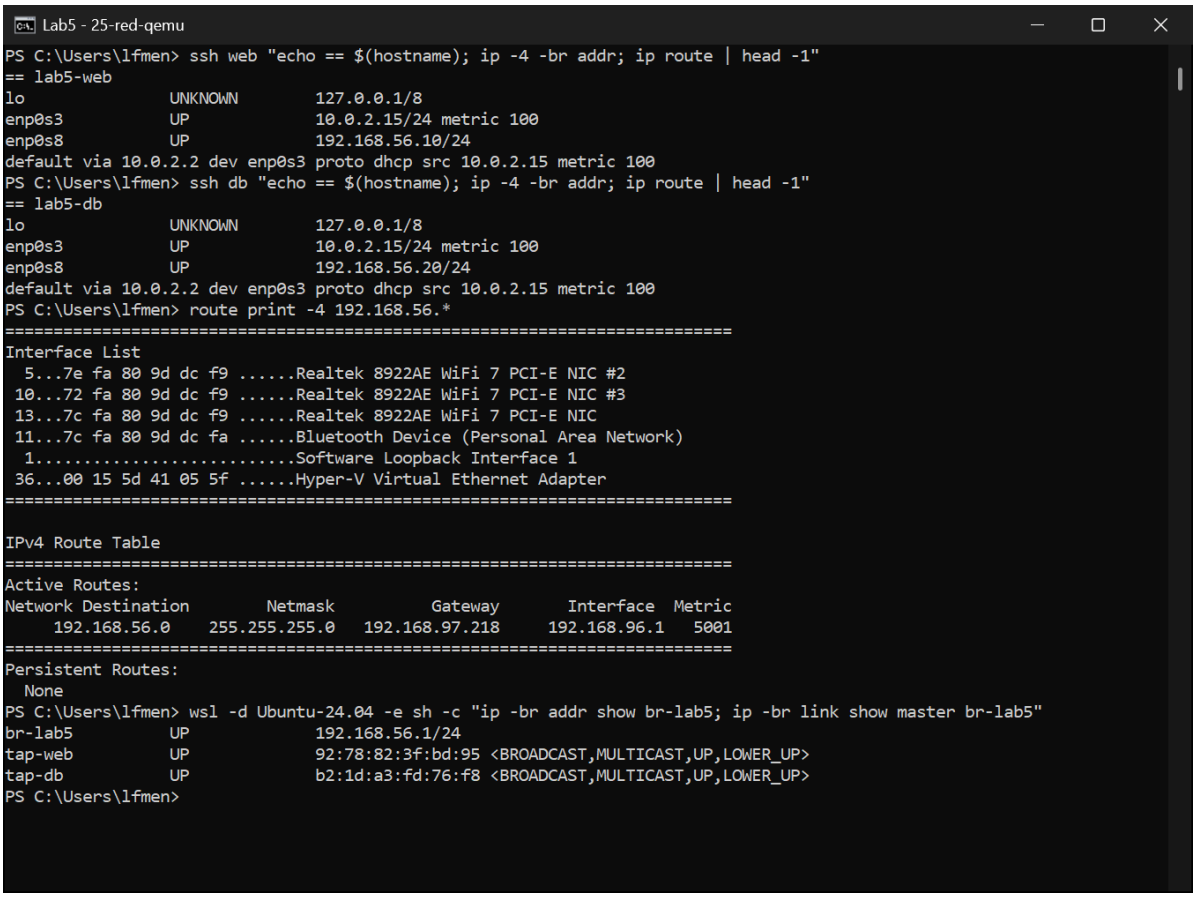


Figura 4. *Interfaces de ambas VM (enp0s3 NAT, enp0s8 red del laboratorio), ruta de Windows hacia WSL y puente br-lab5 con sus interfaces tap.*

4.1 Decisiones de diseño

Tabla 7. *Decisiones de diseño y alternativas.*

Decisión	Alternativa descartada	Motivo
IPs fijas sin DHCP	DHCP en el puente	Firewall, bind-address, wp_user@IP y archivo hosts dependen de direcciones estables
NAT de usuario de QEMU para Internet	Puente a la red física	Las VM no quedan expuestas a la red física; solo tráfico saliente
Interfaces tap propiedad del usuario	Ejecutar QEMU como root	QEMU corre sin privilegios; el root solo crea la red una vez
NIC en ranuras PCI 3 y 8	Renombrar con cloud-init	Nombres enp0s3/enp0s8 estables desde el primer arranque
MASQUERADE de Windows → .1	Abrir UFW a la red de WSL	La red de WSL cambia; .1 es constante y ya está permitida
Imagen cloud + cloud-init	Instalador ISO	Arranque en ~1 min y configuración declarativa
Puertos 22022 y 33306	22 y 3306	Menos escaneos automatizados; complementa llaves y firewall

Decisión	Alternativa descartada	Motivo
Nginx + PHP-FPM	Apache + mod_php	Menor memoria en VM de 2 GB; PHP aislado en su propio proceso
Reto DNS-01 con DuckDNS	HTTP-01, TLS-ALPN-01, autofirmado	Los dos primeros requieren que Let's Encrypt llegue a la VM

5. Procedimiento por etapas

El laboratorio completo se reproduce con tres comandos desde WSL y uno en Windows. Cada script lee los mismos parámetros (`config/lab5.env`) y credenciales (`secretos.env`, no versionado). Las etapas 5.4 a 5.10 se ejecutan dentro de las VM por SSH, con comandos de la forma `ssh web "cd ~/lab5 && sudo bash scripts/<etapa>.sh"`; `infra-qemu/desplegar.sh` las ejecuta en orden y guarda cada salida en `evidencias/qemu/`.

Tabla 8. *Orden de ejecución.*

#	Etapa	Script	Dónde
1	Preparación del host (una vez, con sudo)	infra-qemu/requisitos.sh	WSL
2	Creación y arranque de las VM	infra-qemu/crear-vms.sh	WSL
3	Windows como equipo cliente (administrador)	infra-qemu/windows.ps1	Windows
4	SSH en 22022	scripts/comun/01-ssh-puerto.sh	web y db
5	Firewall UFW	scripts/comun/02-ufw.sh	web y db
6	MySQL	scripts/bd/01-mysql.sh	db
7	Nginx, PHP, cliente MySQL, WP-CLI	scripts/web/01-stack-web.sh	web
8	WordPress y permisos	scripts/web/02-wordpress-permisos.sh	web
9	SFTP para wpadmin	scripts/web/03-sftp.sh	web
10	HTTPS con Let's Encrypt	scripts/web/04-https.sh	web
11	Pruebas	scripts/pruebas/* · desplegar.sh --solo-pruebas	VM, WSL y Windows

5.1 Preparación del host

`requisitos.sh` es el único paso que necesita privilegios de administrador. Instala QEMU, `qemu-utils`, `cloud-image-utils`, `iptables` y `telnet`; agrega al usuario al grupo `kvm` (acceso a `/dev/kvm`) y crea el servicio `lab5-red`, que en cada arranque de WSL levanta el puente `br-lab5` con 192.168.56.1/24, crea `tap-web` y `tap-db` a nombre del usuario, activa el reenvío IPv4, aplica la regla de MASQUERADE para el tráfico que llega desde Windows y agrega la línea del dominio al archivo `/etc/hosts` de WSL.

```
$ sudo bash infra-qemu/requisitos.sh
== Paquetes
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
== Acceso a KVM para lfmendoza
== Red del laboratorio (servicio lab5-red)
Created symlink /etc/systemd/system/multi-user.target.wants/lab5-red.service
active
Listo.
```

Figura 5. *Preparación del host en WSL2.*

5.2 Creación de las máquinas virtuales

Objetivo. Disponer de dos servidores Ubuntu Server 26.04 sin entorno gráfico, conectados entre sí y con el host por una red privada, y con salida a Internet.

Implementación. `crear-vms.sh` descarga la imagen cloud oficial de Ubuntu Server 26.04.1 y crea para cada VM un disco qcow2 de 15 GB que usa la imagen como base (solo guarda las diferencias). Genera un disco *NoCloud* con tres archivos de cloud-init: `user-data` (hostname, zona horaria, usuario `sysadmin` con la llave ed25519 del laboratorio y sin acceso por contraseña), `meta-data` y `network-config`, que identifica las interfaces por MAC y configura `enp0s3` por DHCP con DNS 1.1.1.1 y 8.8.8.8 y `enp0s8` con IP fija. Cada VM arranca como proceso de QEMU con 2 vCPU, 2 GB de RAM, `accel=kvm`, `-display none` y la salida serie en un archivo; el apagado se hace de forma ordenada por QMP. Al final agrega los alias `web` y `db` a `~/.ssh/config` y espera a que el login por SSH funcione.

```
$ infra-qemu/crear-vms.sh
lab5-web arrancada (192.168.56.10)
lab5-db arrancada (192.168.56.20)
Esperando a que SSH responda (primer arranque ~1 min)...
lab5-web
lab5-db
$ ssh web "lsb_release -d; cloud-init status"
Description:    Ubuntu 26.04.1 LTS
status: done
```

Figura 6. *Creación de las VM y primer acceso por SSH.*

5.3 Windows como equipo cliente

`windows.ps1`, ejecutado como administrador, agrega la ruta `192.168.56.0/24` hacia la IP de WSL, escribe la línea del dominio en `C:\Windows\System32\drivers\etc\hosts`, copia la llave del laboratorio con permisos restringidos y crea los alias `web` y `db` en el `~/.ssh/config` de Windows, y habilita el cliente telnet. FileZilla se instaló a partir del archivo portable oficial, después de verificar su suma SHA-512 contra la publicada por el proyecto.

5.4 Acceso SSH y puerto no conocido

Decisión. Autenticación solo con llave ed25519 y puerto 22022. La llave elimina los ataques de fuerza bruta contra contraseñas; el puerto no conocido reduce el ruido de escáneres automatizados que prueban el 22. Es una medida de oscuridad que complementa a la llave y al firewall, no los reemplaza.

Implementación. `01-ssh-puerto.sh` crea `/etc/ssh/sshd_config.d/10-lab5.conf` con `Port 22022`, `PermitRootLogin no`, `PasswordAuthentication no`, `KbdInteractiveAuthentication no` y `X11Forwarding no`. Antes de aplicar el cambio valida la sintaxis con `sshd -t` para no perder el acceso remoto. En Ubuntu 24.04 y posteriores el puerto lo abre systemd (`ssh.socket`), por lo que el script ejecuta `systemctl daemon-reload` y reinicia el socket. Después se cambió el puerto de los alias a 22022.

```
$ ssh web "cd ~/lab5 && sudo bash scripts/comun/01-ssh-puerto.sh"
LISTEN 0  4096       0.0.0.0:22022    0.0.0.0:*
users:(("sshd",pid=2060,fd=3),("systemd",pid=1,fd=72))
LISTEN 0  4096          [::]:22022       [::]:*
users:(("sshd",pid=2060,fd=4),("systemd",pid=1,fd=74))
OK: SSH escuchando en 22022
$ ssh -p 22 sysadmin@192.168.56.10
ssh: connect to host 192.168.56.10 port 22: Connection refused
$ ssh -p 22 sysadmin@192.168.56.20
ssh: connect to host 192.168.56.20 port 22: Connection refused
```

Figura 7. *SSH en 22022; antes de activar UFW, el 22 responde con rechazo.*

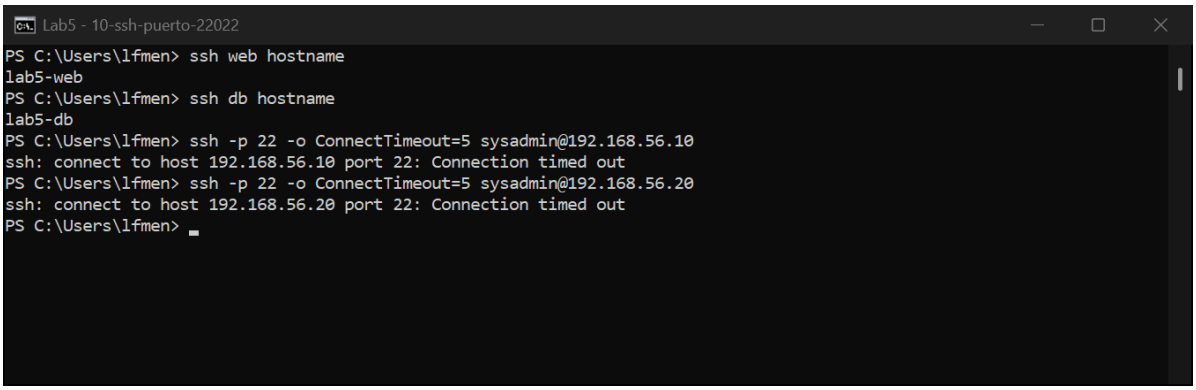


Figura 8. *Con UFW activo, SSH entra por 22022 y el puerto 22 ya no responde (los paquetes se descartan).*

5.5 Firewall UFW

Decisión. Política por defecto de denegar todo el tráfico entrante y permitir el saliente, con reglas por servicio y por origen. SSH solo se acepta desde la red del laboratorio. El 80 queda abierto únicamente para redirigir a HTTPS. En la VM de datos, el 33306 solo admite al servidor web (192.168.56.10): ni siquiera el host llega a MySQL. Esta regla y el `bind-address` de MySQL se cubren mutuamente. Con UFW activo, un puerto cerrado no responde (DROP), en lugar de rechazar la conexión, lo que da menos información a quien escanea.

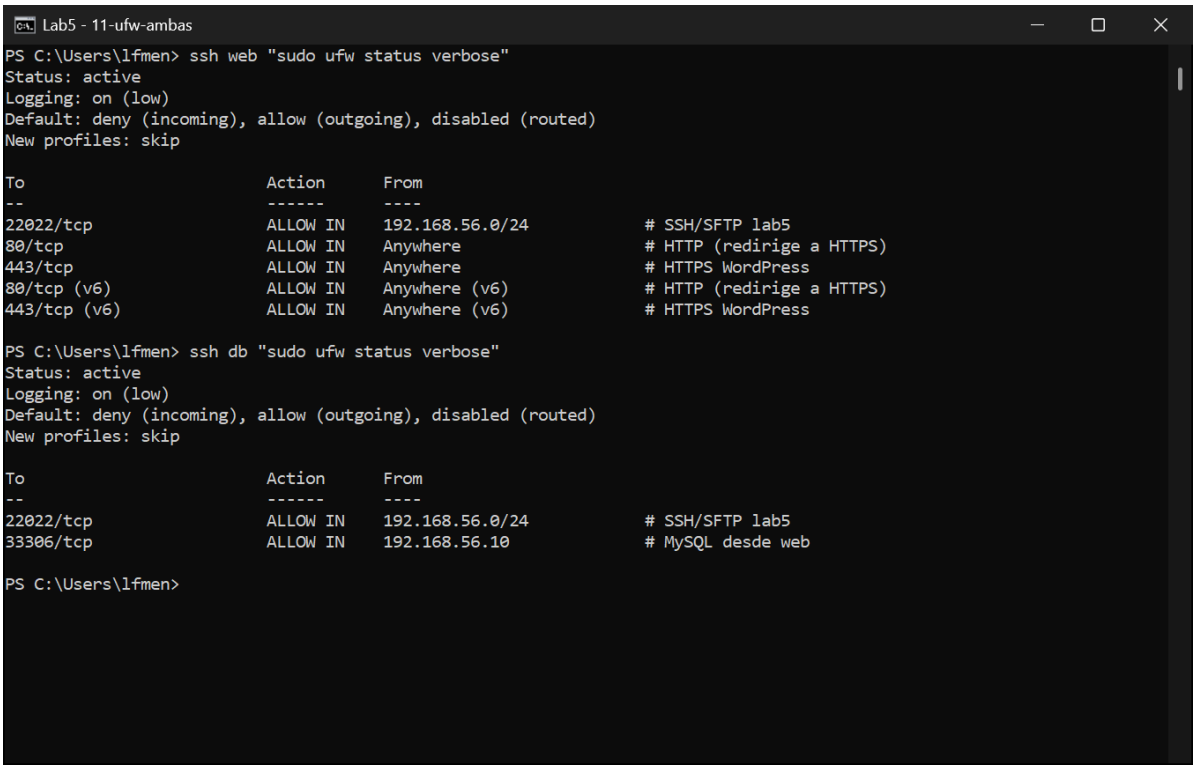


Figura 9. *Reglas de UFW en ambas VM.*

5.6 Capa de base de datos

Decisión. MySQL 8.4 LTS. El usuario `wp_user` solo puede conectarse desde 192.168.56.10 y solo tiene privilegios sobre `wordpress_db`, por lo que no es el administrador del motor. El usuario `root` de MySQL conserva `auth_socket`: solo entra quien ya es root en la VM (`sudo mysql`), no tiene contraseña que filtrar y no acepta conexiones remotas.

Implementación. `01-mysql.sh` instala `mysql-server` y crea `/etc/mysql/mysql.conf.d/zz-lab5.cnf` con `bind-address = 192.168.56.20`, `port = 33306` y `mysqlx = 0`. El prefijo `zz-` es necesario porque MySQL lee esos archivos en orden alfabético y conserva el último valor; con otro nombre, `mysqld.cnf` restablece `127.0.0.1`. Luego crea la base en `utf8mb4` y el usuario con `CREATE USER IF NOT EXISTS` + `ALTER USER`, lo que hace al script idempotente.

```
$ ssh db "cd ~/lab5 && sudo bash scripts/bd/01-mysql.sh"
user    host    plugin
wp_user 192.168.56.10   caching_sha2_password
GRANT USAGE ON *.* TO `wp_user`@`192.168.56.10`
GRANT ALL PRIVILEGES ON `wordpress_db`.* TO `wp_user`@`192.168.56.10`
mysql  Ver 8.4.11-0ubuntu0.26.04.1 for Linux on x86_64 ((Ubuntu))
LISTEN 0      151    192.168.56.20:33306      0.0.0.0:*    users:(("mysqld",pid=3178,fd=18))
```

Figura 10. *Instalación de MySQL: usuario, privilegios y puerto.*

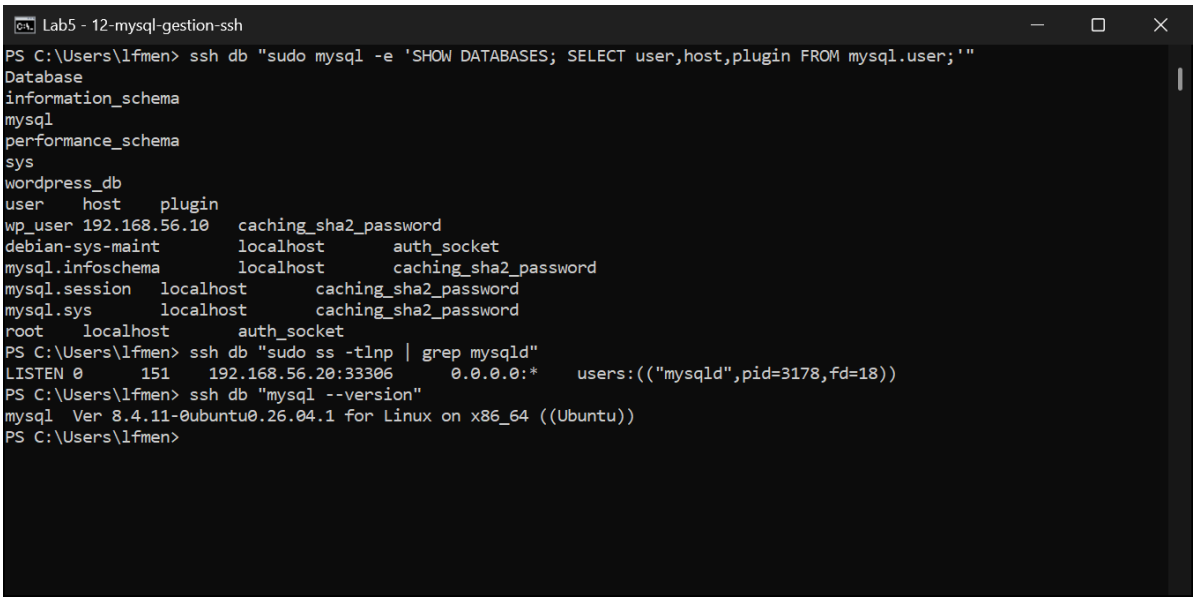


Figura 11. *Gestión de la base por SSH: bases, usuarios con su autenticación y MySQL escuchando solo en 192.168.56.20:33306.*

5.7 Capa web

`01-stack-web.sh` instala Nginx, PHP-FPM con las extensiones de WordPress (`mysql`, `curl`, `gd`, `mbstring`, `xml`, `zip`, `intl`, `imagick`), el cliente `mysql` (requisito) y WP-CLI, elimina el sitio por defecto y oculta la versión de Nginx. `02-wordpress-permisos.sh` prueba primero la conexión remota con `wp_user` y después instala WordPress con WP-CLI apuntando a `192.168.56.20:33306`. En `wp-config.php` se fijan `DISALLOW_FILE_EDIT`, `DISALLOW_FILE_MODS` y `FORCE_SSL_ADMIN`: el panel no intenta escribir en disco y el inicio de sesión siempre usa HTTPS.

```
$ ssh web "cd ~/lab5 && sudo bash scripts/web/02-wordpress-permisos.sh"
estado
OK conexión a BD remota
Downloading WordPress 7.1.4 (en_US)...
md5 hash verified: e3ab64c0de73fe22c0e482c00fa9298c
Success: Generated 'wp-config.php' file.
Success: Added the constant 'DISALLOW_FILE_MODS' to the 'wp-config.php' file with the raw value
'true'.
Success: WordPress installed successfully.
Success: Language activated.
```

Figura 12. *Instalación de WordPress contra la base remota.*

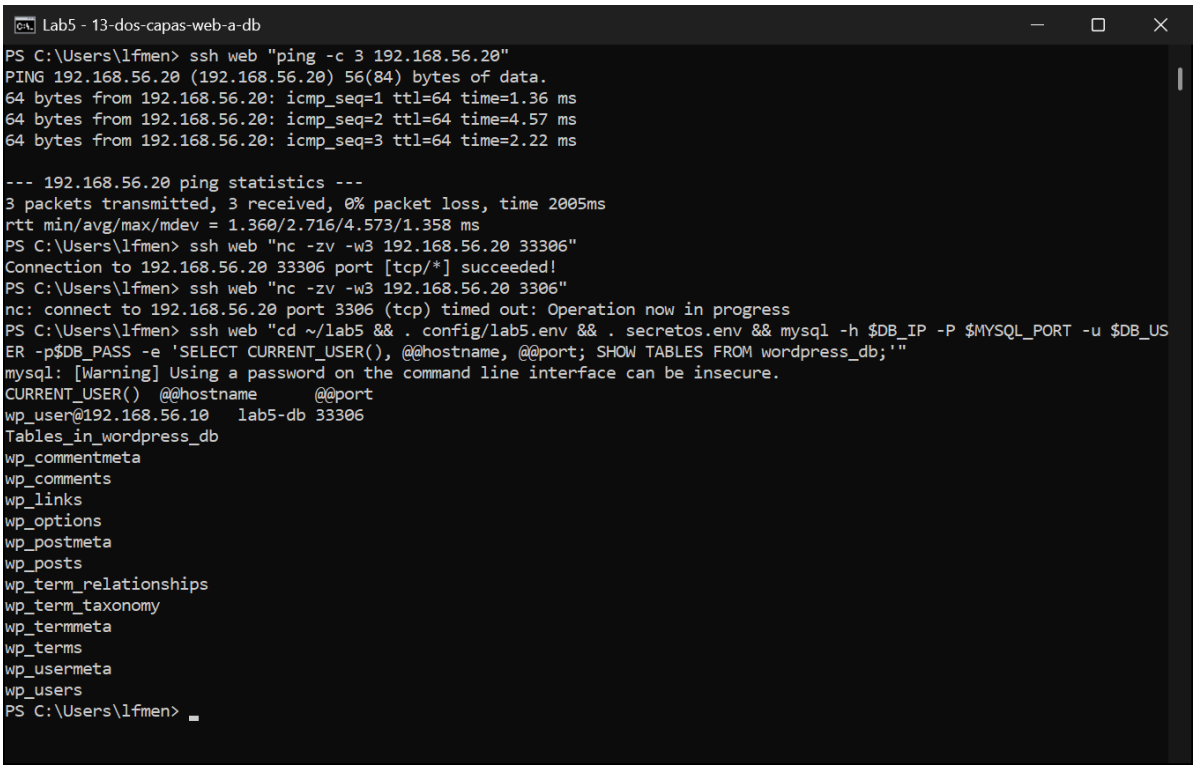


Figura 13. *Despliegue en 2 capas: ping, puerto 33306 abierto, 3306 cerrado y login remoto de wp_user con las tablas de WordPress.*

5.8 Permisos con chown y chmod

Cada archivo tiene propietario, grupo y permisos para tres niveles (propietario, grupo, otros), combinando lectura (4), escritura (2) y ejecución (1; en directorios, poder entrar). `chown` cambia propietario y grupo; `chmod`, los permisos.

Diseño. `wpadmin`, sin shell, es el único usuario con escritura. El grupo es `www-data`, con el que corren Nginx y PHP-FPM: el servicio lee, pero no modifica. Los demás usuarios, incluido `sysadmin`, no tienen permisos. Si se explotara WordPress, el código del atacante correría como `www-data` y no podría alterar archivos; `wp-config.php`, que guarda la contraseña de la base, tampoco es legible por otros.

Tabla 9. *Modelo de permisos del sitio.*

Ruta	Propietario:grupo	Modo	Significado
/var/www	root:root	755	Lo exige el chroot de SFTP: la jaula debe pertenecer a root
/var/www/wordpress y subdirectorios	wpadmin:www-data	2750	rwx wpadmin, r-x www-data, nada otros; setgid: lo nuevo hereda el grupo www-data
Archivos	wpadmin:www-data	640	rw- wpadmin, r-- www-data, nada otros

```
chown -R wpadmin:www-data /var/www/wordpress
find /var/www/wordpress -type d -exec chmod 2750 {} +
find /var/www/wordpress -type f -exec chmod 640 {} +
```

Figura 14. *Comandos aplicados por 02-wordpress-permisos.sh.*

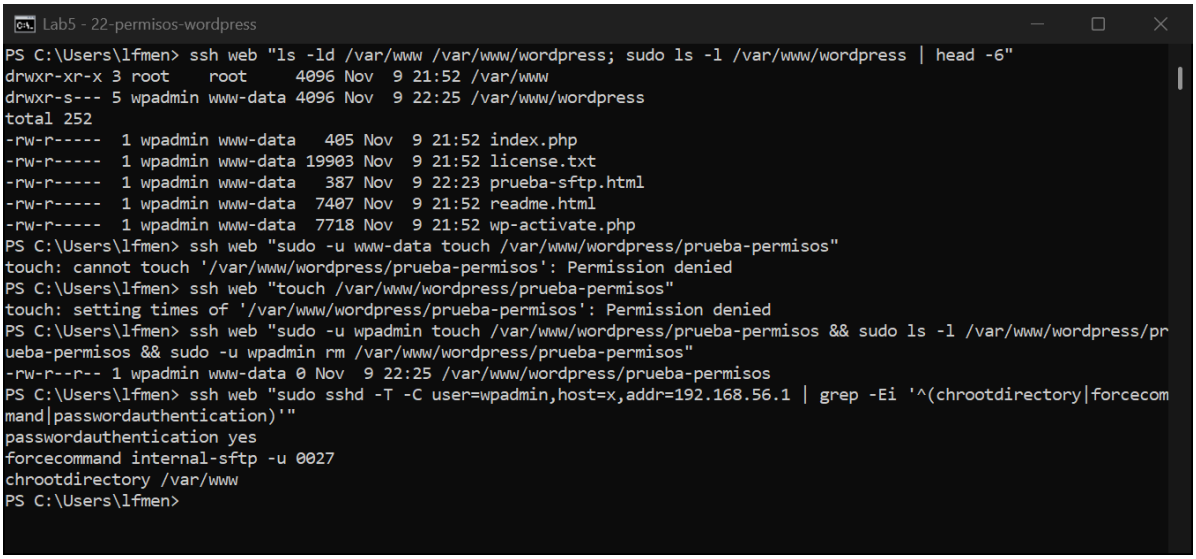


Figura 15. *Permisos resultantes; www-data y sysadmin reciben Permission denied y solo wpadmin escribe. Configuración efectiva de SFTP para wpadmin.*

5.9 SFTP y FileZilla

Decisión. SFTP viaja cifrado dentro de SSH, sin servidor FTP adicional ni puertos extra. `03-sftp.sh` agrega un bloque `Match User wpadmin` con `ChrootDirectory /var/www` (wpadmin solo ve `/wordpress`), `ForceCommand internal-sftp -u 0027` (sin terminal; lo subido queda 640/750), contraseña habilitada solo para él y túneles deshabilitados. `sysadmin` sigue entrando únicamente con llave.

En la primera conexión FileZilla muestra la huella de la llave del servidor. Antes de aceptarla se comparó con la huella real de `lab5-web` (`ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`, `SHA256:XX4MpdWZ…UukE`), que es el paso que evita un ataque de intermediario.

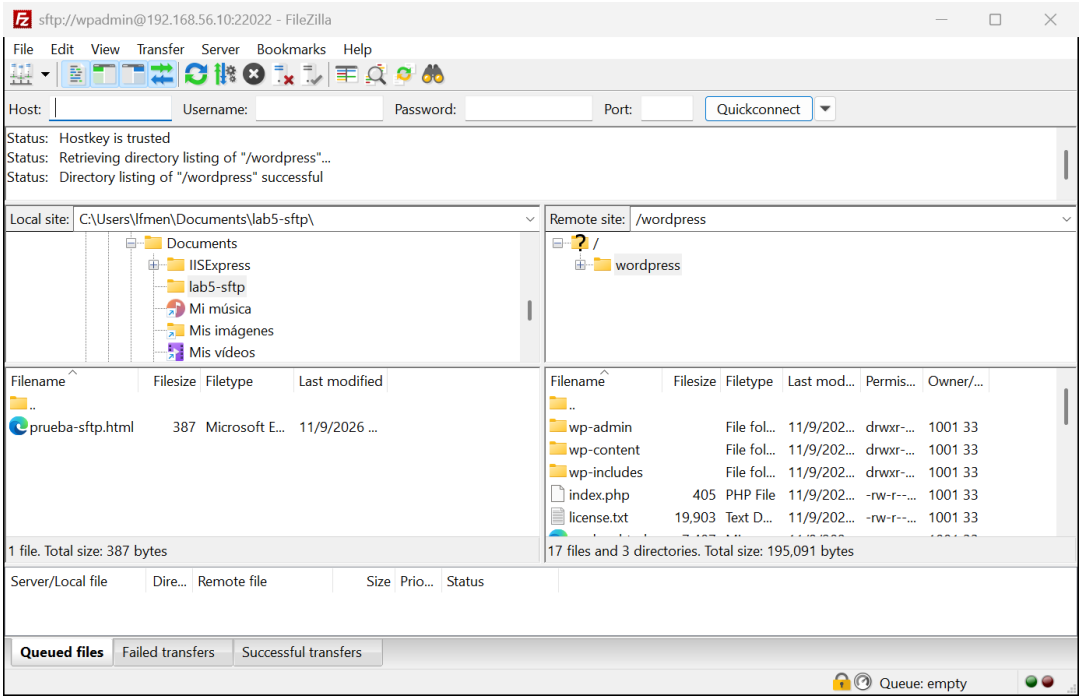


Figura 16. *FileZilla conectado como wpadmin a 192.168.56.10:22022: la llave del servidor es de confianza y la jaula solo muestra /wordpress.*

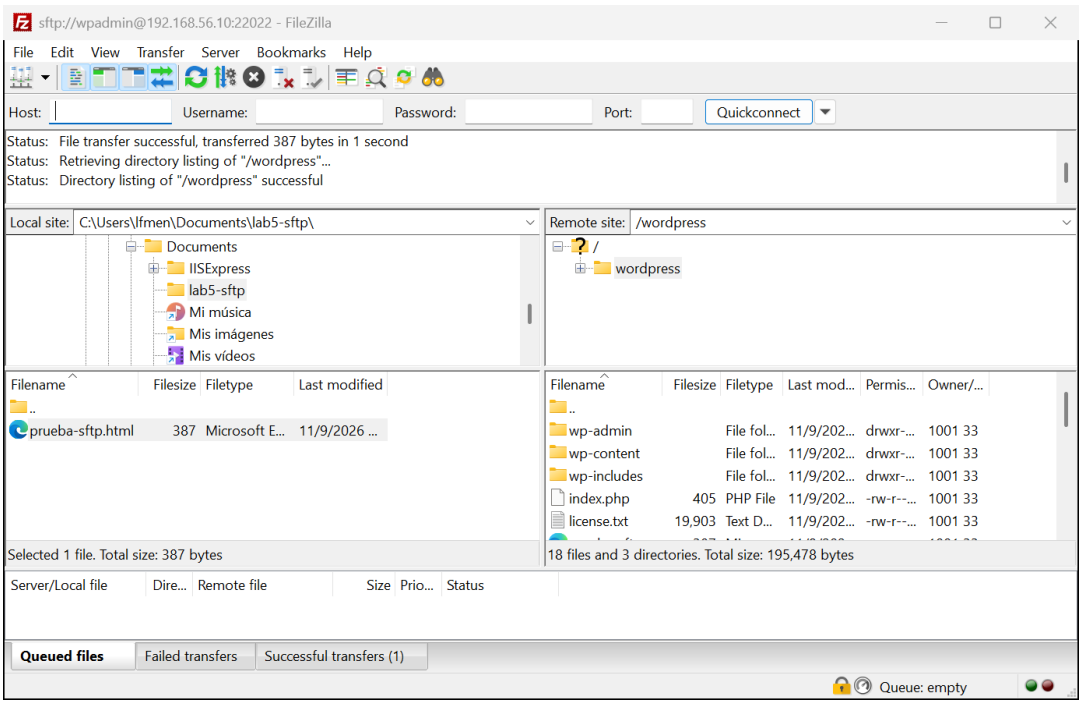


Figura 17. *Subida de prueba-sftp.html: File transfer successful.*

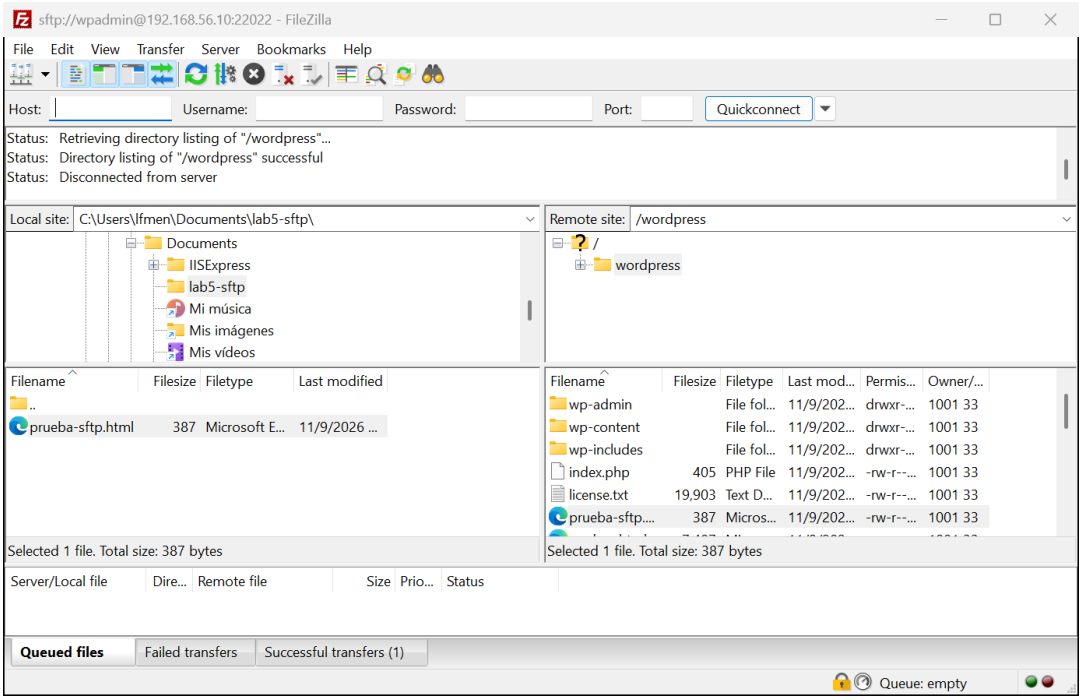


Figura 18. *El archivo en el servidor con permisos -rw-r----- y dueño wpadmin (1001) grupo www-data (33).*



Figura 19. *El archivo subido se sirve por HTTPS: Nginx lo lee gracias al grupo www-data.*

5.10 HTTPS con Let's Encrypt

Let's Encrypt solo emite certificados a quien demuestra control sobre el dominio. Evaluamos sus tres métodos de validación:

Tabla 10. *Métodos de validación de Let's Encrypt.*

Método	Cómo valida	¿Aplica a este diseño?
HTTP-01	Descarga un archivo desde el puerto 80 del servidor	No: la VM no es accesible desde Internet
TLS-ALPN-01	Negocia TLS con el servidor en el puerto 443	No: mismo motivo
DNS-01	Consulta un registro TXT en el DNS público del dominio	Sí: no necesita conectarse al servidor

Decisión. DNS-01 con el subdominio gratuito `redes-lab5-wp.duckdns.org`, porque DuckDNS ofrece una API para publicar el registro TXT. Un dominio inventado (`.local`, `.lab`) no existe en el DNS público y no puede validarse. El nombre existe públicamente solo para la validación; el archivo hosts del host lo dirige a la IP privada. El navegador acepta el certificado porque verifica el nombre y la cadena de confianza, no la IP a la que resolvió.

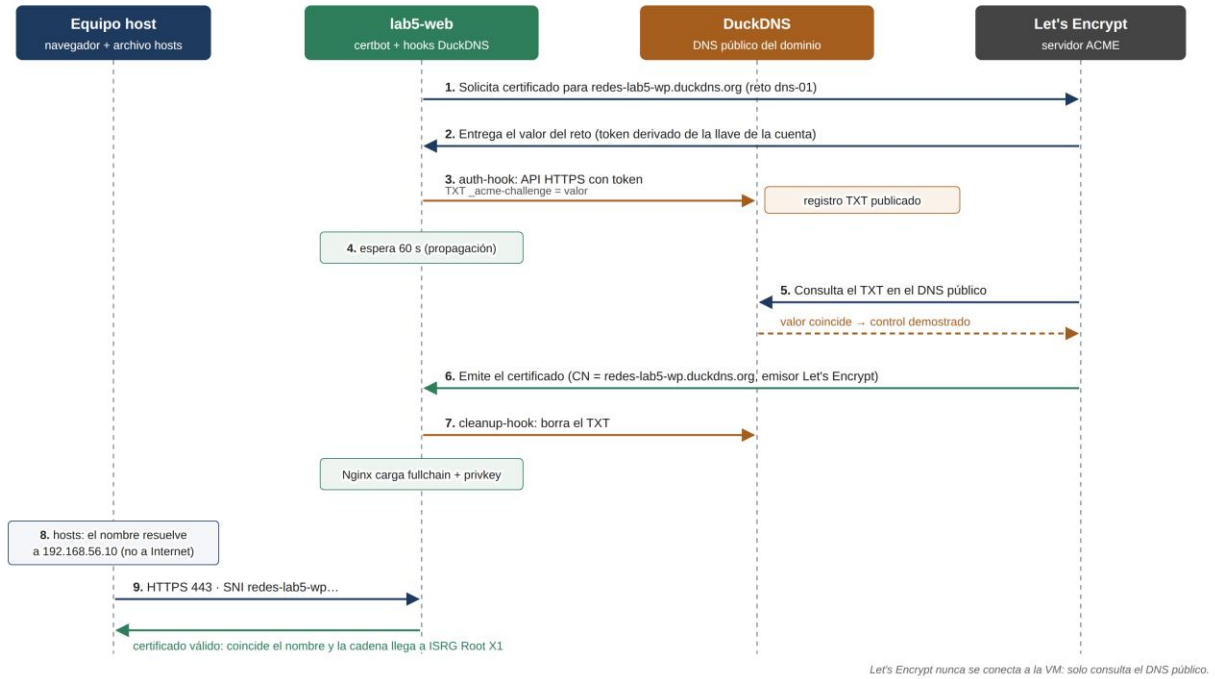


Figura 20. *Secuencia de emisión con DNS-01 y uso del certificado desde el host.*

Implementación. `04-https.sh` instala certbot con dos *hooks*: uno publica el valor del reto en DuckDNS y espera 60 s a que se propague; el otro lo borra al terminar. El token se guarda en `/etc/letsencrypt/duckdns/credenciales` con permisos 600. Primero se validó contra el entorno de pruebas (*staging*) para no consumir el límite de 5 certificados idénticos por semana. Nginx quedó con redirección 301 de HTTP a HTTPS, TLS 1.2 y 1.3, HTTP/2, HSTS y bloqueo de `wp-config.php`, `xmlrpc.php` y archivos ocultos. `certbot.timer` renueva el certificado automáticamente.

```
$ ssh web "cd ~/lab5 && sudo bash scripts/web/04-https.sh"
Requesting a certificate for redes-lab5-wp.duckdns.org
Hook '--manual-auth-hook' for redes-lab5-wp.duckdns.org ran with output:
 DuckDNS: OK
Successfully received certificate.
This certificate expires on 2027-02-08.
nginx: configuration file /etc/nginx/nginx.conf test is successful
issuer=C=US, O=Let's Encrypt, CN=YE1
subject=CN=redes-lab5-wp.duckdns.org
```

Figura 21. *Emisión del certificado en lab5-web.*

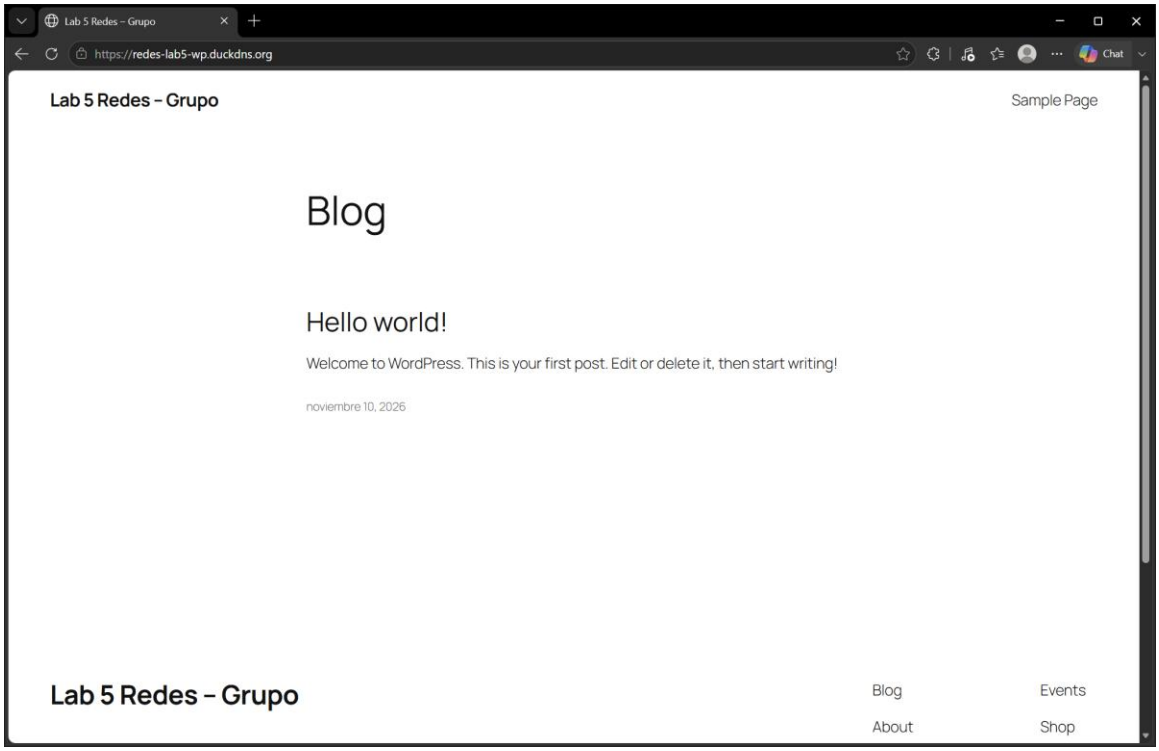


Figura 22. *WordPress servido por HTTPS en el navegador del host, sin advertencias.*

5.11 Resolución de nombres local

El archivo `C:\Windows\System32\drivers\etc\hosts` contiene la línea `192.168.56.10 redes-lab5-wp.duckdns.org`. El resolver del sistema operativo consulta ese archivo antes que a los servidores DNS, por lo que el nombre resuelve a la VM web sin depender de Internet. En WSL el servicio `lab5-red` agrega la misma línea a `/etc/hosts`, porque WSL regenera ese archivo en cada arranque.

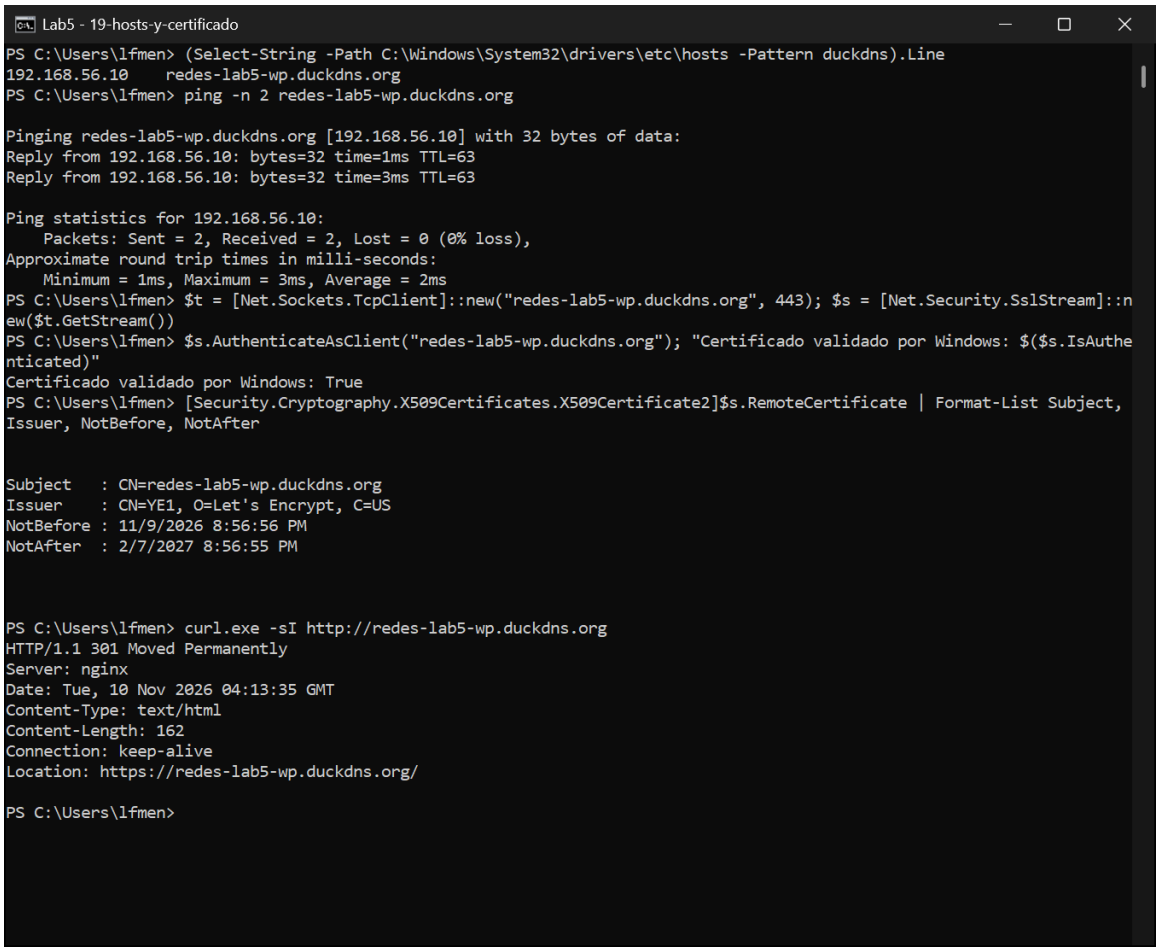


Figura 23. *Archivo hosts, ping al dominio (192.168.56.10), certificado validado por Windows (Let's Encrypt YE1, válido hasta el 07/02/2027 hora local) y redirección 301 de HTTP a HTTPS.*

6. Cliente y servidor

Los términos se usan en dos sentidos. Como **aplicación**, el servidor es el programa que ofrece un servicio y el cliente el que lo consume. Como **ordenador**, son las máquinas que cumplen esos papeles. Una misma máquina puede ser servidor en una conexión y cliente en otra: `lab5-web` es servidor para el navegador y cliente de la base de datos.

Tabla 11. *Roles de cliente y servidor en cada conexión.*

Conexión	Aplicación cliente → servidor	Ordenador cliente → servidor	Protocolo / puerto
Sitio web	Edge → Nginx + PHP-FPM	Host → lab5-web	HTTPS 443
Archivos del sitio	FileZilla → OpenSSH (internal-sftp)	Host → lab5-web	SFTP 22022
Administración	ssh → sshd	Host → lab5-web y lab5-db	SSH 22022
Datos de WordPress	PHP (mysqli) y cliente mysql → mysqld	lab5-web → lab5-db	MySQL 33306
Certificado	certbot → Let's Encrypt y API DuckDNS	lab5-web → servidores en Internet	HTTPS 443 (NAT)
Virtualización	QEMU (proceso de usuario) → KVM (kernel)	WSL2 → hardware vía Hyper-V	—

7. Seguridad: defensa en profundidad

Ningún control se considera suficiente por sí solo. La tabla resume las capas aplicadas y el riesgo que mitiga cada una.

Tabla 12. *Controles de seguridad aplicados.*

Control	Ámbito	Riesgo que mitiga
Llave ed25519, sin contraseñas ni login de root por SSH	SSH	Fuerza bruta y robo de contraseñas
SSH en 22022 y MySQL en 33306	Red	Escaneos y ataques automatizados a puertos conocidos
UFW con denegación por defecto (DROP) y reglas por origen	Red	Exposición de servicios no previstos; reconocimiento
bind-address en la red privada + regla 33306 solo desde .10	Base de datos	Acceso directo a MySQL desde otros equipos
wp_user limitado a wordpress_db; root con auth_socket	Base de datos	Escalamiento si se filtra wp-config.php
wpadmin único con escritura; www-data solo lectura	Archivos	Modificación del sitio tras explotar WordPress
SFTP enjaulado con ForceCommand y sin túneles	SFTP	Navegación del sistema y uso como shell o proxy

Control	Ámbito	Riesgo que mitiga
Verificación de la huella de la llave del servidor	SFTP/SSH	Ataque de intermediario en la primera conexión
DISALLOW_FILE_MODS y DISALLOW_FILE_EDIT	WordPress	Instalación de plugins o edición de código desde el panel
TLS 1.2/1.3, HSTS y redirección 80 → 443	Transporte	Intercepción y degradación a HTTP
Bloqueo de wp-config.php, xmlrpc.php y ocultos; server_tokens off	Nginx	Fuga de información y fuerza bruta por XML-RPC
QEMU sin privilegios; NAT de usuario solo saliente	Hipervisor	Impacto de una falla en QEMU; exposición a la red física
secretos.env fuera del repositorio; token DuckDNS con permisos 600	Operación	Filtración de credenciales

8. Pruebas de funcionamiento

Las pruebas están automatizadas en `scripts/pruebas/`: `pruebas-servidor.sh` dentro de cada VM, `pruebas-host-mac-linux.sh` desde WSL y `pruebas-host-windows.ps1` desde Windows. `desplegar.sh --solo-pruebas` las ejecuta todas y guarda los resultados. Cada requisito se verifica junto con su caso negativo.

Tabla 13. *Matriz de pruebas.*

Requisito	Prueba	Resultado esperado	Resultado obtenido
Conectividad entre capas	ping web → db y db → web	0 % de pérdida	0 % (Fig. 13)
Puerto de MySQL desde web	nc -zv 192.168.56.20 33306	Conecta	succeeded (Fig. 13)
Puerto por defecto cerrado	nc -zv 192.168.56.20 3306	Sin respuesta	timed out (Fig. 13)
Login remoto (2 capas)	mysql -h 192.168.56.20 -P 33306 -u wp_user	Usuario, host y tablas wp_*	wp_user@192.168.56.10, lab5-db, 33306 (Fig. 13)
telnet web → MySQL	telnet 192.168.56.20 33306 desde lab5-web	Saludo del servidor	8.4.11 (Fig. 24)
telnet host → HTTPS	telnet 192.168.56.10 443	Conecta	Sesión abierta (Fig. 26)
telnet host → MySQL	telnet 192.168.56.20 33306	No conecta	Connect failed (Fig. 25)
Puertos desde el host	Test-NetConnection 22022, 443, 22, 33306	True, True, False, False	Cumple (Fig. 27)
Gestión remota y SO	ssh web / ssh db con lsb_release	Ubuntu 26.04.1 LTS	Cumple (Fig. 27)
Permisos	touch como www-data, sysadmin y wpadmin	Solo wpadmin escribe	Cumple (Fig. 15)
SFTP	FileZilla sube un archivo y se abre en el sitio	Archivo visible por HTTPS	Cumple (Fig. 17-19)
HTTPS válido	curl sin -k; validación TLS de Windows	200 y emisor Let's Encrypt	HTTP 200; IsAuthenticated True (Fig. 23)
Redirección HTTP	curl -I http://redes-lab5-wp.duckdns.org	301 a https://	Cumple (Fig. 23)
Resolución local	hosts + ping al dominio	192.168.56.10	Cumple (Fig. 23)



Figura 24. *telnet desde lab5-web al 33306: MySQL responde con su saludo (8.4.11, caching_sha2_password).*

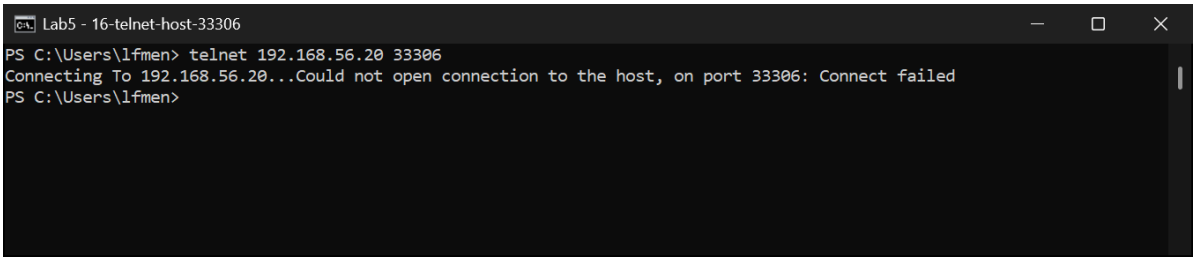


Figura 25. *telnet desde el host al 33306: Connect failed; UFW solo permite a 192.168.56.10.*



Figura 26. *telnet desde el host al 443: la conexión se abre (pantalla en negro).*

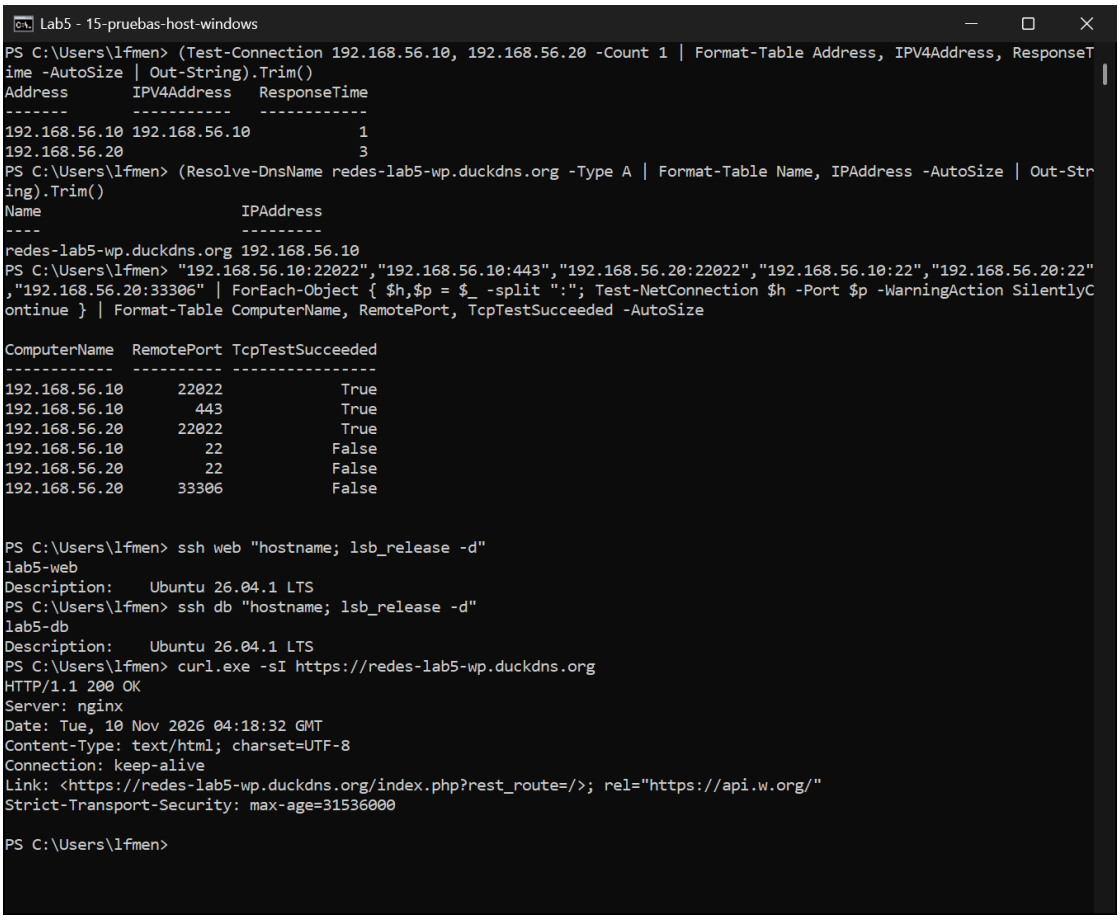


Figura 27. *Pruebas desde el host Windows: ping, resolución local, puertos abiertos y cerrados, SSH y HTTPS.*

```
$ infra-qemu/desplegar.sh --solo-pruebas      (extracto de evidencias/qemu/pruebas-host-wsl.txt)
===== Puertos abiertos =====
Connection to 192.168.56.10 22022 port [tcp/*] succeeded!
Connection to 192.168.56.10 443 port [tcp/https] succeeded!
Connection to 192.168.56.20 22022 port [tcp/*] succeeded!
===== Puertos CERRADOS =====
nc: connect to 192.168.56.10 port 22 (tcp) timed out: Operation now in progress
nc: connect to 192.168.56.20 port 22 (tcp) timed out: Operation now in progress
nc: connect to 192.168.56.20 port 33306 (tcp) timed out: Operation now in progress
===== HTTPS (sin -k: el certificado debe ser válido) =====
HTTP/2 200
===== HTTP redirige a HTTPS =====
HTTP/1.1 301 Moved Permanently
```

Figura 28. *Pruebas desde WSL (cliente Linux) con los mismos resultados.*

9. Problemas encontrados y soluciones

Tabla 14. *Problemas y soluciones.*

Problema	Causa	Solución
La NIC NAT quedó como enp0s2 en el primer arranque	cloud-init no puede renombrar una interfaz que ya está activa	Colocar las NIC en las ranuras PCI 3 y 8: Ubuntu las nombra enp0s3 y enp0s8 sin renombrar
ssh rechazado con «System is booting up»	El puerto abre antes de terminar el arranque (pam_nologin)	Esperar a que el login funcione, no solo a que el puerto abra
route add fallaba desde PowerShell	PowerShell 5.1 trata como error el stderr de comandos nativos y altera las comillas al llamar a wsl.exe	Ejecutar route por cmd y obtener la IP de WSL con hostname -I
SSH desde Windows bloqueado por UFW	El tráfico llegaba con la IP de WSL, fuera de 192.168.56.0/24	MASQUERADE en WSL: entra a las VM como 192.168.56.1
FileZilla no se instala con winget	No está en el catálogo de winget	Archivo portable oficial, verificado con SHA-512
telnet no mostraba la versión de MySQL	El byte de versión del protocolo (10) es un salto de línea y head cortaba antes	Mostrar más líneas del saludo
ssh -p 22 da «timed out» y no «refused»	Con UFW activo los paquetes se descartan (DROP)	Comportamiento esperado; «refused» se observa antes de activar UFW
MySQL escucha en 127.0.0.1 si el archivo no es el último	MySQL lee mysql.conf.d en orden alfabético	Nombre zz-lab5.cnf; verificar con my_print_defaults mysqld
El puerto de SSH no cambia al reiniciar ssh	Desde Ubuntu 24.04 el puerto lo abre ssh.socket	systemctl daemon-reload y reinicio de ssh.socket

10. Conclusiones

1. QEMU/KVM cumplió todos los requisitos del laboratorio con menos fricción que un hipervisor de escritorio en un equipo con WSL2: las VM son procesos sin pantalla que se crean, arrancan y apagan desde la línea de comandos, y la imagen cloud con cloud-init redujo la instalación a un minuto por servidor.
2. Separar aplicación y datos solo reduce la superficie de ataque si se acompaña de controles de red: con `bind-address`, la regla de UFW y un usuario limitado a su base, MySQL no es alcanzable ni desde el host y una filtración de `wp-config.php` no compromete el motor.
3. El modelo propietario/grupo/otros separa a quien administra el contenido (wpadmin) de quien lo sirve (www-data). Con esa separación, una vulnerabilidad en WordPress no basta para modificar el sitio.
4. El reto DNS-01 resuelve el conflicto aparente entre un certificado público y un servidor privado: Let's Encrypt valida el dominio en el DNS público y el archivo hosts, que tiene prioridad sobre el DNS, lleva el nombre a la IP interna sin afectar la validez del certificado.

5. Administrar solo por SSH obliga a automatizar. Con los scripts versionados el laboratorio completo se reconstruye con tres comandos, y cada salida queda registrada como evidencia.

11. Referencias

[1] QEMU Project, "QEMU System Emulation User's Guide: Network emulation, Invocation", versión 8.2. https://www.qemu.org/docs/master/

[2] Linux Kernel Documentation, "KVM: The Kernel Virtual Machine". https://docs.kernel.org/virt/kvm/

[3] Microsoft, "WSL documentation: Advanced settings configuration; Networking considerations", 2026. https://learn.microsoft.com/windows/wsl/

[4] Canonical, "cloud-init documentation: NoCloud datasource; Networking config version 2". https://docs.cloud-init.io/

[5] Canonical, "Ubuntu Server documentation: OpenSSH, Firewall (UFW) y Netplan", 2026. https://documentation.ubuntu.com/server/

[6] Oracle, "MySQL 8.4 Reference Manual: Using Option Files; Server System Variables". https://dev.mysql.com/doc/refman/8.4/en/

[7] OpenBSD, "sshd_config(5): Match, ChrootDirectory, ForceCommand". https://man.openbsd.org/sshd_config

[8] F5/NGINX, "Module ngx_http_ssl_module" y "ngx_http_core_module". https://nginx.org/en/docs/

[9] Let's Encrypt, "Challenge Types" y "Rate Limits". https://letsencrypt.org/docs/

[10] R. Barnes et al., "Automatic Certificate Management Environment (ACME)", RFC 8555, IETF, 2019.

[11] Electronic Frontier Foundation, "Certbot User Guide: manual plugin and hooks". https://eff-certbot.readthedocs.io/

[12] DuckDNS, "Spec: update API and TXT records". https://www.duckdns.org/spec.jsp

[13] WordPress.org, "Hardening WordPress" y "WP-CLI Commands". https://developer.wordpress.org/

[14] Linux man-pages, chmod(1), chown(1), hosts(5), ip-link(8), iptables(8), ufw(8).

Anexo A. Reproducción del laboratorio

Requisitos: Windows 11 con WSL2 (Ubuntu 24.04) y virtualización anidada disponible (`/dev/kvm`), y el archivo `secretos.env` en la raíz del repositorio. Las recetas de `just` agrupan los pasos:

```
# WSL, una sola vez por equipo (pide sudo)
$ just -f infra-qemu/justfile requisitos
# Crear y arrancar las VM
$ just -f infra-qemu/justfile vms
# Desplegar todo el laboratorio y registrar evidencias en evidencias/qemu/
$ just -f infra-qemu/justfile desplegar --certificado
# Windows, PowerShell como administrador (repetir si cambia la IP de WSL)
PS> powershell -ExecutionPolicy Bypass -File infra-qemu\windows.ps1
# Solo pruebas / apagar / eliminar
$ just -f infra-qemu/justfile pruebas
$ just -f infra-qemu/justfile apagar
$ just -f infra-qemu/justfile borrar
```

Figura 29. *Comandos para reconstruir el laboratorio.*

Tabla 15. *Archivos del repositorio usados en esta implementación.*

Archivo	Función
config/lab5.env	IPs, puertos, usuarios y dominio comunes a todos los scripts
secretos.env (no versionado)	Contraseñas y token de DuckDNS; plantilla en secretos.env.example
infra-qemu/requisitos.sh, red.sh	Paquetes, grupo kvm y servicio lab5-red (puente, tap, NAT, hosts)
infra-qemu/crear-vms.sh	Imagen cloud, discos qcow2, cloud-init y arranque de las VM con KVM
infra-qemu/desplegar.sh, justfile	Despliegue completo por SSH y registro de evidencias
infra-qemu/windows.ps1	Ruta, archivo hosts, llave y alias SSH, telnet en Windows
infra-qemu/capturas/	Automatización de las capturas de pantalla (solo la ventana de cada aplicación)
scripts/comun, bd, web	Configuración de SSH, UFW, MySQL, capa web, permisos, SFTP y HTTPS
scripts/pruebas/*	Pruebas dentro de las VM y desde el host
evidencias/qemu/	Salidas registradas (.txt) y capturas (capturas/)